process you would never write a test that you expect to pass before writing more code. In practice, though, you may do this all the time. Sometimes I see and create an abstraction in the code, but there are still valid test cases to write. In particular, I'll often write code for potential error conditions even if I think they are already covered in the code. It's a balance because you lose some of the benefit of TDD by creating too many test cases that don't drive code changes. One way to keep the balance is to make a list of the test cases before you start writing the tests—that way you'll remember to cover all the interesting cases.

And some things are just hard. In particular, some parts of your application will be dependent on an external piece of code in a way that makes it difficult to isolate them for unit testing. *Test doubles*, which are special kinds of objects that can stand in for objects that are part of your application, are one way to work around this issue; see Chapter 7, *Using Test Doubles as Mocks and Stubs*, on page 129. But there are definitely cases (though they're not common) in which the cost of testing a complex feature is higher than the value of the tests.

Recent discussions in the Rails community have debated whether TDD's design benefits are even valuable. You may have heard the phrase "test-driven design damage." I strongly believe that TDD, and the relatively smaller and more numerous classes that a TDD process often brings, do result in more clear and more valuable code. But the TDD process is not a replacement for good design instincts; it's still possible to create bad code when testing, or even to create bad code in the name of testing.

Words to Live By

- Any change to the program logic should be driven by a failed test.
- If it's not tested, treat it like it's broken.
- Testing is supposed to help for the long term. The long term starts tomorrow, or maybe after lunch.
- It's not done until it works.
- Tests are code that doesn't have tests; keep them simple.
- Start a bug fix by writing a test.
- Tests monitor the quality of your codebase. If it becomes difficult to write tests, that often means your codebase is too interdependent.